

Agenda:

1. Number of 1 Bits
2. Finding Good Days
3. Single Number II
4. Subarrays with OR 1

“
Behind every high achiever is a story of hard work and countless study sessions. Be the author of your success.
”

Number of 1 Bits

Problem Description

Write a function that takes an **integer** and returns the **number of 1 bits** present in its binary representation.

Problem Constraints

$1 \leq A \leq 10^9$

$N = 5 \rightarrow 101$ $Ans = \underline{2}$

cnt = 0

for i → 0 to 31 {

TC = $O(31) = O(1)$

if ((A & (1 << i)) > 0) cnt++

SC = $O(1)$

}

return cnt

$A \gg 3$

A = 1101001

$A \gg 3 \rightarrow 0001101$

$(A \gg i) \& 1 \rightarrow 1 \Rightarrow i^{\text{th}} \text{ bit} = 1$
 $\rightarrow 0 \Rightarrow i^{\text{th}} \text{ bit} = 0$

cnt = 0

while (A > 0) {

if ((A & 1) > 0) cnt++

$A \% 2$

A = A >> 1

$A / 2$

}

return cnt

$A \rightarrow \frac{A}{2} \rightarrow \frac{A}{2^2} \rightarrow \frac{A}{2^3} \dots \rightarrow \frac{A}{2^K} = 1$

TC = $O(\log_2(A))$

SC = $O(1)$

$\Rightarrow A = 2^K$

$\Rightarrow K = \underline{\log_2(A)}$

Finding Good Days

Problem Description

Alex has a cat named Boomer. He decides to put his cat to the test for eternity.

He starts on day 1 with one stash of food unit, every next day, the stash doubles.

If Boomer is well behaved during a particular day, only then she receives food worth equal to the stash produced on that day.

Boomer receives a net worth of A units of food. What is the number of days she received the stash?

Problem Constraints

$$1 \leq A \leq 2^{31}-1$$

day $\rightarrow$	1	2	3	4	5	6	
	1	2	4	8	16	32	...
	2^0	2^1	2^2	2^3	2^4	2^5	

$$A = 13$$

$$\text{Ans} = 3$$

$$A = 20 \rightarrow \text{days} \rightarrow \{3, 5\} \quad \text{Ans} = 2$$
$$4 + 16 = 20$$

$$\text{Ans} = \text{\# of set bits in } A$$

Single Number II

Problem Description

Given an array of integers, every element appears thrice except for one, which occurs once.

Find that element that does not appear thrice.

NOTE: Your algorithm should have a linear runtime complexity.

Could you implement it without using extra memory?

Problem Constraints

- $2 \leq A \leq 5 \cdot 10^6$
- $0 \leq A[i] \leq \text{INTMAX}$

$A = [4, 5, 5, 1, 4, 5, 4]$

Ans = 1

20 min

Bruteforce → $\forall A[i]$, calculate frequency.

Element with freq = 1 is answer.

→ Travel for each element → $TC = O(N^2)$

$SC = O(1)$

→ calculate frequency in Map → $TC = O(N)$

$SC = O(N)$

$$A = [\overset{0}{5} \ \overset{1}{7} \ \overset{2}{5} \ \overset{3}{9} \ \overset{4}{6} \ \overset{5}{7} \ \overset{6}{5} \ \overset{7}{9} \ \overset{8}{9} \ \overset{9}{7}]$$

	3	2	1	0
5	0	1	0	1
7	0	1	1	1
5	0	1	0	1
9	1	0	0	1
6	0	1	1	0
7	0	1	1	1
5	0	1	0	1
9	1	0	0	1
9	1	0	0	1
7	0	1	1	1

bits $\rightarrow$ 3 7 4 9
0 1 1 0
6 (Ans)

TC = $O(31 * N) = \underline{O(N)}$

SC = $O(1)$

count of bits at i^{th}
 position $\rightarrow 3 * K \Rightarrow i^{th}$ bit = 0
 for answer

$3 * K + 1 \Rightarrow i^{th}$ bit = 1
 for answer

```

ans = 0
for i  $\rightarrow$  0 to 31 { //  $\forall$  bits
  cnt = 0
  for j  $\rightarrow$  0 to (N-1) {
    if ((A[j] & (1 << i)) > 0) cnt++
    // check ith bit = 1
  }
  if (cnt % 3 == 1) {
    ans = ans | (1 << i)
  }
}
  
```

Subarrays with OR 1

Problem Description

You are given a binary array A of length N where each element is either 0 or 1.

Your task is to count the number of subarrays where the bitwise OR of all the elements in the subarray is 1.

Problem Constraints

$1 \leq N \leq 10^4$

A[i] is either 0 or 1

0 1 2 3
A = [1 0 0 1]
Ans = 7

20 min

Brute force → $\forall$ subarrays calculate OR
& count # subarrays with OR = 1.

TC = $O(N^3)$ → $O(N^2)$ carry forward

SC = $O(1)$

Total # of subarrays = $N * \frac{(N+1)}{2}$ ✓

Total subarrays = # subarrays with OR = 1 + # subarrays with OR = 0



→ If all elements are 0 then only subarray OR = 0.

$$A = [\overset{0}{0} \overset{1}{0} \overset{2}{0} \overset{3}{1} \overset{4}{0} \overset{5}{0} \overset{6}{1} \overset{7}{1} \overset{8}{1} \overset{9}{0} \overset{10}{1} \overset{11}{1} \overset{12}{0} \overset{13}{0} \overset{14}{0} \overset{15}{0} \overset{16}{1}]$$

$$\downarrow$$

$$\frac{3 * (3+1)}{2} = 6$$

ans = 0

cnt = 0

for i → 0 to (N-1) {

if (A[i] == 0) cnt++

else { ans += cnt * (cnt+1) / 2

cnt = 0

}

}

ans += cnt * (cnt+1) / 2

return N * (N+1) / 2 - ans

TC = O(N)

SC = O(1)

$$A = [\overset{0}{0} \overset{1}{0} \overset{2}{0} \overset{3}{1} \overset{4}{0} \overset{5}{0} \overset{6}{1} \overset{7}{1} \overset{8}{1} \overset{9}{0} \overset{10}{1} \overset{11}{1} \overset{12}{0} \overset{13}{0} \overset{14}{0} \overset{15}{0} \overset{16}{1}]$$

$$ans = 0 + 3 * 4 / 2 = 6 + 2 * 3 / 2 = 9 + 1 * 2 / 2 = 10 + 4 * 5 / 2$$

$$cnt = 0 + 2 \ 3 \ 0 + 2 \ 0 + 0 + 2 \ 3 \ 4 \ 0$$

$$= \underline{\underline{20}}$$

$$\text{Ans} = \frac{17 \times 18^9}{2} - 20 = 153 - 20 = \underline{\underline{133}}$$